

LES JOINTURES

- INTRODUCTION
- JOINTURE INTERNE
- JOINTURE EXTERNE
- JOINTURE EXTERNE GAUCHE
- JOINTURE EXTERNE DROITE
- JOINTURE AVEC USING
- JOINTURE NATURELLE
- AUTO-JOINTURE

Introduction

- ❖ Une **jointure** est une opération permettant de combiner les données de plusieurs tables en fonction d'une colonne commune. Elle est utilisée pour récupérer des informations réparties sur plusieurs tables.
- ❖ Une jointure est faite sur la base d'une certaine relation appelée **condition de jointure**
- ❖ La **condition de jointure** est souvent établie **entre des colonnes définies comme clé primaire et clé étrangère**.
- ❖ Une jointure peut avoir plusieurs conditions de jointure, ceci dépendant du nombre de tables impliquées dans la jointure.

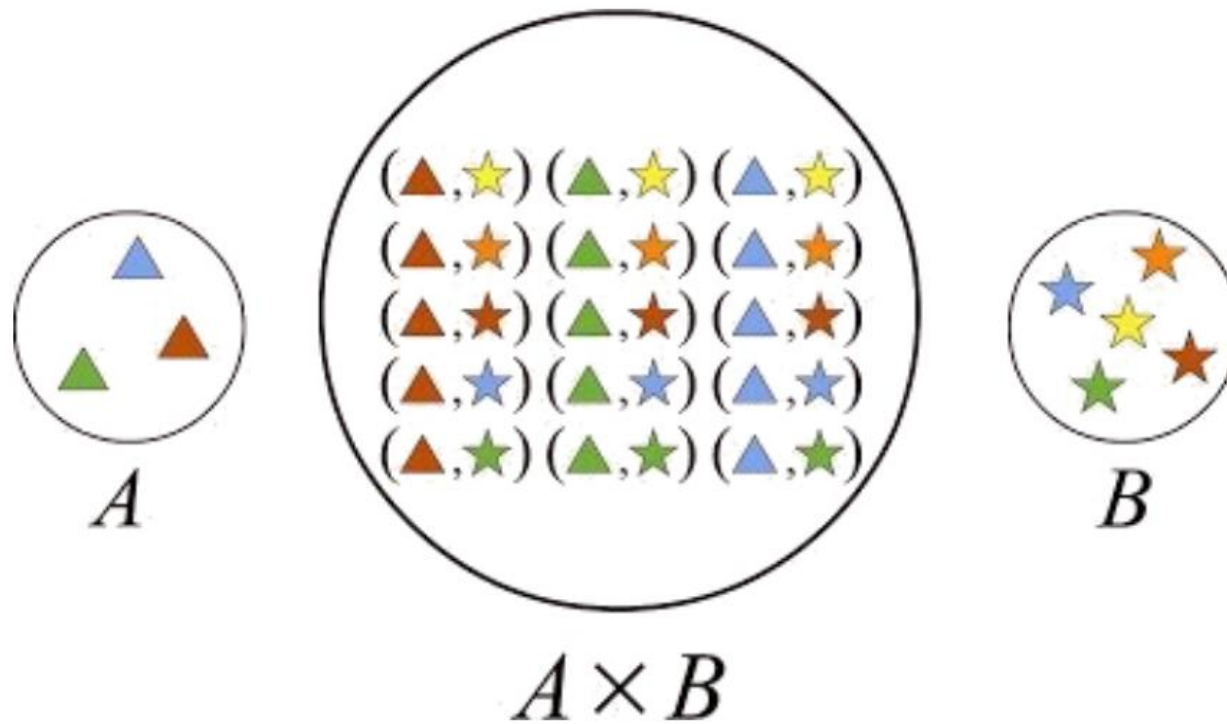
Introduction

❖ NOTEZ BIEN

- Lorsqu'on a une requête supposée de « jointure » qui vérifie l'une des situations ci-dessous, il ne s'agit plus d'une **jointure** mais d'un **produit cartésien**
 - ✓ Une condition de jointure est omise,
 - ✓ Une condition de jointure est invalide,
 - ✓ Tous les enregistrements de la première table sont liés à tous les enregistrements de la seconde table.
- Le **produit cartésien** génère un grand nombre d'enregistrements dont le résultat est **rarement très utile**.

Introduction

❖ Illustration d'un produit cartésien



Introduction

❖ Il existe plusieurs types de jointures:

- Jointure interne
- Jointure externe
- Jointure naturelle
- Auto-jointure

Jointure interne

- ❖ Une **jointure interne** est une opération qui récupère uniquement les enregistrements ayant une correspondance dans les deux tables.
- ❖ Elle sélectionne uniquement les lignes où la **condition de jointure** est **satisfaite dans les deux tables**.
- ❖ Elle est introduite par les mots clés **JOIN... ON...** ou **INNER JOIN ... ON ...**
 - **ON** permet de spécifier la **condition de jointure**
 - **INNER** est facultatif (on peut utiliser **INNER JOIN** ou **JOIN**)

Jointure interne

❖ Syntaxe SQL2:

```
SELECT * | [liste_colonne]  
FROM table1  
[INNER] JOIN table2 ON condition_jointure;  
[WHERE ...]
```

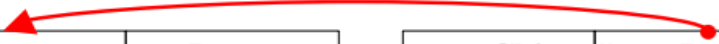
❖ Syntaxe SQL1:

```
SELECT table1.col, table2.col  
FROM table1, table2  
WHERE condition_jointure;
```

Ici la condition de jointure est précisée dans la clause **WHERE**

Jointure interne

❖ On considère les tables **Personne** et **Chien** ci-dessous



<u>nomPersonne</u>	agePersonne
Tim	12
John	23
Bill	30
Paul	50

<u>nomChien</u>	#nomPersonne	poids	ageChien
Pollux	Paul	12	5
Nestor	Tim	20	10
Rex	Bill	18	15
Boby	Tim	10	8

❖ **Ex1:** Requête qui liste les personnes avec leurs chiens

```
SELECT P.nomPersonne, C.nomChien
```

```
FROM Personne P
```

```
INNER JOIN Chien C ON P.nomPersonne=C.nomPersonne; (SQL2)
```

```
SELECT P.nomPersonne, C.nomChien
```

```
FROM Personne P, Chien C
```

```
WHERE P.nomPersonne=C.nomPersonne; (SQL1)
```

Jointure interne

❖ **Ex2:** Requête qui liste les personnes avec leurs chien si le chien à plus de 10 ans

```
SELECT P.nomPersonne, C.nomChien  
FROM Personne P  
JOIN Chien C ON P.nomPersonne=C.nomPersonne  
WHERE C.ageChien > 10; (SQL2)
```

```
SELECT P.nomPersonne, C.nomChien  
FROM Personne P, Chien C  
WHERE P.nomPersonne=C.nomPersonne  
AND C.ageChien > 10; (SQL1)
```

<u>nomPersonne</u>	agePersonne
Tim	12
John	23
Bill	30
Paul	50

<u>nomChien</u>	#nomPersonne	poids	ageChien
Pollux	Paul	12	5
Nestor	Tim	20	10
Rex	Bill	18	15
Boby	Tim	10	8

Jointure interne

❖ **Ex3:** Requête qui liste des personnes avec le poids total de leur(s) chien(s) (par personne)

```
SELECT P.nomPersonne, SUM(poids) AS PoidsTotal  
FROM Personne P  
INNER JOIN Chien C ON P.nomPersonne=C.nomPersonne  
GROUP BY P.nomPersonne; (SQL2)
```

```
SELECT P.nomPersonne, SUM(poids) AS PoidsTotal  
FROM Personne P  
WHERE P.nomPersonne=C.nomPersonne  
GROUP BY P.nomPersonne; (SQL1)
```

<u>nomPersonne</u>	agePersonne
Tim	12
John	23
Bill	30
Paul	50

<u>nomChien</u>	#nomPersonne	poids	ageChien
Pollux	Paul	12	5
Nestor	Tim	20	10
Rex	Bill	18	15
Boby	Tim	10	8

Jointure externe

- ❖ **Une jointure externe** est une jointure qui récupère **toutes les lignes d'une table**, même si aucune correspondance n'est trouvée dans l'autre table. Contrairement à une jointure interne qui ne **renvoie que les correspondances**
- ❖ La jointure externe garantit que les données de l'une des tables soient entièrement conservées avec des valeurs NULL pour les colonnes de la table sans correspondance
- ❖ Il existe deux types de jointures externes:
 - **La jointure par la droite**
 - **La jointure par la gauche**

Jointure externe gauche

- ❖ **Une jointure externe par la gauche** récupère toutes les lignes de la table de gauche (sauf restrictions dans une clause WHERE bien sûr), même si certaines n'ont pas de correspondance dans la table de droite
- ❖ Elle est introduite par les mots clés **LEFT JOIN ... ON ...** ou **LEFT OUTER JOIN ... ON ...**
- ❖ **OUTER** est facultatif, à condition d'utiliser **LEFT**
- ❖ La **table de gauche** est la première table mentionnée dans la requête, c'est-à-dire, en général, la table donnée dans la clause **FROM**
- ❖ La **table de droite** est la table mentionnée dans la clause **LEFT JOIN**

Jointure externe gauche

❖ **Syntaxe:**

SELECT * | [liste_colonnes]

FROM table1

// table1 est la table de gauche

LEFT [OUTER] JOIN table2 **ON** condition_jointure;

[WHERE ...]

❖ **OUTER JOIN** retourne les tuples qui remplissent la condition de la jointure, mais aussi certains tuples qui ne la satisfont pas

❖ **LEFT** récupère tous les tuples de la table de gauche, ceux sans correspondance dans l'autre table sont inclus dans le résultat avec un NULL pour les attributs de l'autre table

Jointure externe gauche

❖ On considère les tables **Eleve**, **Universite**, et **Candidature** ci-dessous

idE	nomE	moyenneLycée	effectifLycée
123	Ana	19.5	1000
234	Bob	18	1500
345	Chloé	17.5	500
456	Damien	19.5	1000
543	Chloé	17	2000
567	Éléonore	14.5	2000
654	Ana	19.5	1000
678	Farid	19	200
765	Joana	14.5	1500
789	Gisèle	17	800
876	Irène	19.5	400
898	Hector	18.5	800

Table ÉLÈVE

nomU	ville	effectif
INSA	Lyon	36000
UCB	Lyon	15000
UJF	Grenoble	10000
UJM	Saint-Étienne	21000

Table UNIVERSITÉ

idE	nomU	département	décision
123	INSA	informatique	O
123	UCB	électronique	N
123	UCB	informatique	O
123	UJM	électronique	O
234	INSA	biologie	N
345	UJF	bioinformatique	O
345	UJM	bioinformatique	N
345	UJM	électronique	N
345	UJM	informatique	O
543	UJF	informatique	N
678	UCB	histoire	O
765	UCB	histoire	O
765	UJM	histoire	N
765	UJM	psychologie	O
876	UCB	informatique	N
876	UJF	biologie	O
876	UJF	biologie marine	N
898	INSA	informatique	O
898	UCB	informatique	O

Table CANDIDATURE

Jointure externe gauche

❖ **Ex:** Les élèves avec une moyenne supérieure à 19 et les éventuelles universités où ils/elles ont candidaté

SELECT DISTINCT e.idE, nomE, nomU

FROM Eleve e

LEFT OUTER JOIN Candidature c **ON** c.idE = e.idE

WHERE moyenneLycee > 19;

idE	nomE	nomU
123	Ana	INSA
123	Ana	UCB
123	Ana	UJM
456	Damien	NULL
654	Ana	NULL
876	Irène	UCB
876	Irène	UJF

Jointure externe droite

- ❖ **Une jointure externe par la droite** récupère toutes les lignes de la table de droite (sauf restrictions dans une clause **WHERE** bien sûr), même si certaines n'ont pas de correspondance dans la table de gauche
- ❖ Elle est introduite par les mots clés **RIGHT JOIN ... ON ...** ou **RIGHT OUTER JOIN ... ON ...**
- ❖ **OUTER** est facultatif, à condition d'utiliser **RIGHT**

Jointure externe droite

❖ Syntaxe:

SELECT * | [liste_colonnes]

FROM table1

RIGHT [OUTER] JOIN table2 **ON** condition_jointure;

[WHERE ...]

❖ **table2** est la table de droite

❖ **RIGHT** récupère **tous les tuples de la table de droite**, ceux sans correspondance dans l'autre table sont inclus dans le résultat avec un NULL pour les attributs de l'autre table

Jointure externe droite

❖ **Ex:** Les élèves avec une moyenne supérieure à 19 et les éventuelles universités où ils/elles ont candidaté

```
SELECT DISTINCT e.idE, nomE, nomU  
FROM Candidature c  
RIGHT OUTER JOIN ON Eleve e c.idE = e.idE  
WHERE moyenneLycee > 19;
```

idE	nomE	nomU
123	Ana	INSA
123	Ana	UCB
123	Ana	UJM
456	Damien	NULL
654	Ana	NULL
876	Irène	UCB
876	Irène	UJF

Jointure avec USING

❖ Lorsque les colonnes qui servent à joindre les deux tables **ont le même nom**, on peut utiliser la clause **USING** au lieu de la clause **ON**

❖ **Syntaxe:**

```
SELECT * | [liste_colonnes]
```

```
FROM table1
```

```
[INNER | OUTER | LEFT | RIGHT] JOIN table2 USING (col_com);
```

Est équivalent à

```
SELECT *
```

```
FROM table1
```

```
[INNER | OUTER | LEFT | RIGHT] JOIN table2 ON table1.col_com=table2.col_com;
```

Jointure Naturelle

- ❖ Les **jointures naturelles**, comme pour les jointures avec **USING**, peuvent être utilisées uniquement dans le cas où les colonnes servant à la jointure ont le même nom dans les deux tables.
- ❖ Simplement, dans le cas d'une **jointure naturelle**, on ne donne pas la (les) colonne(s) sur laquelle (lesquelles) joindre les tables : c'est déterminé automatiquement.

Jointure Naturelle

❖ Donc si on a les 2 tables suivantes : table1: colonnes A, B, C et table2: colonnes B, C, E

```
SELECT *  
FROM table1
```

```
NATURAL JOIN table2; Est équivalent à  
SELECT *  
FROM table1  
JOIN table2 USING (B);
```

```
Est équivalent à  
SELECT *  
FROM table1  
JOIN table2 ON table1.B=table2.B;
```

Auto-jointure

❖ Une **auto-jointure** est une jointure où une table est jointe à elle-même. Cela permet de comparer des lignes dans la même table, ce qui peut être utile pour des requêtes impliquant des relations hiérarchiques ou des comparaisons entre des enregistrements.

❖ **Ex1:** Liste des employés avec le nom de leur supérieur

```
SELECT e1.nom AS employe, e2.nom AS superieur  
FROM Employes e1  
LEFT JOIN Employes e2 ON e1.superieur_id = e2.id;
```

❖ **Ex2:** Liste des employés originaires de la même région que 'Dupont Jean'

```
SELECT e2.nom AS employe  
FROM Employes e1  
JOIN Employes e2 ON e1.region = e2.region  
WHERE e1.nom = 'Dupont Jean' AND e2.nom != 'Dupont Jean';
```